

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
## Display all the columns of the dataframe

pd.pandas.set_option('display.max_columns',None)
```

```
dataset=pd.read_csv('/content/sample_data/train.csv') # Changed the path here
```

```
## print shape of dataset with rows and columns
print(dataset.shape)
```

```
(1460, 81)
```

```
dataset.head()
```

```
(1460, 81)
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	LotConfig	LandSlope	Neighborhood	C
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	Inside	Gtl	CollgCr	
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	FR2	Gtl	Veenker	
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	Inside	Gtl	CollgCr	
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	Corner	Gtl	Crawfor	
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	FR2	Gtl	NoRidge	

```
features_with_na=[features for features in dataset.columns if dataset[features].isnull().sum()>1]
## 2- step print the feature name and the percentage of missing values
```

```
for feature in features_with_na:
    print(feature, np.round(dataset[feature].isnull().mean(), 4), ' % missing values')
```

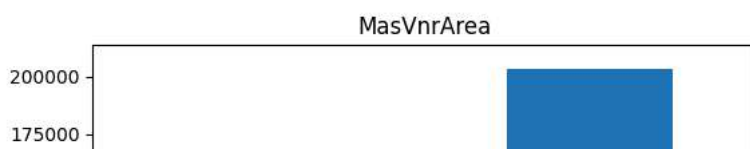
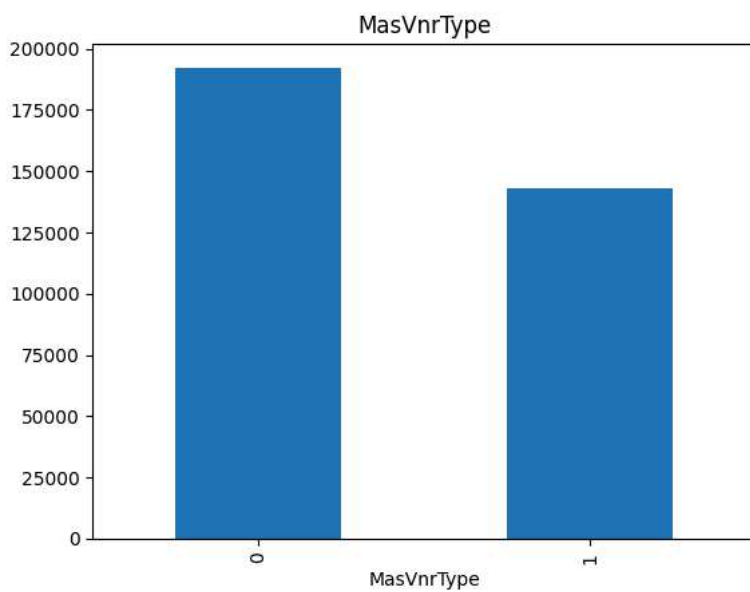
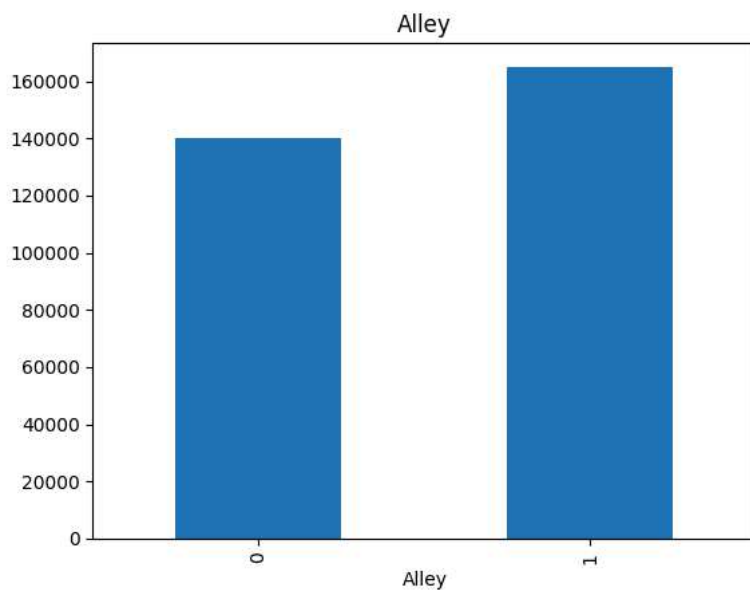
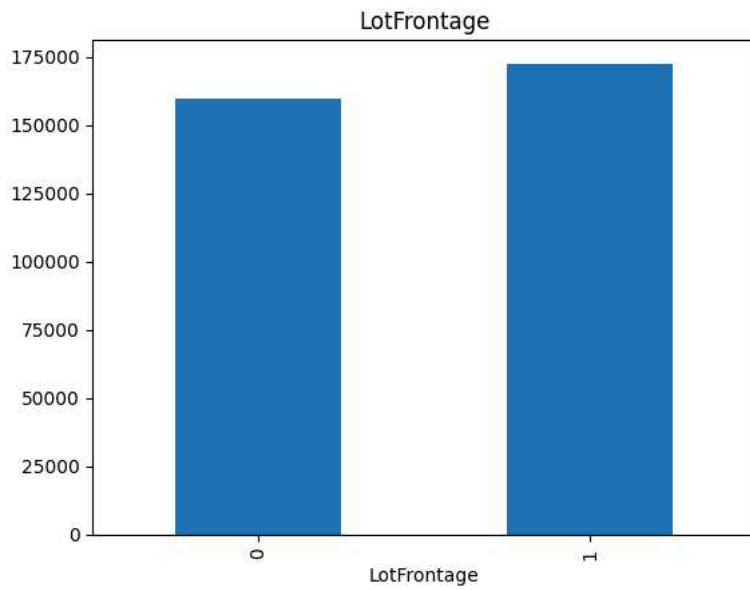
```
(1460, 81)
```

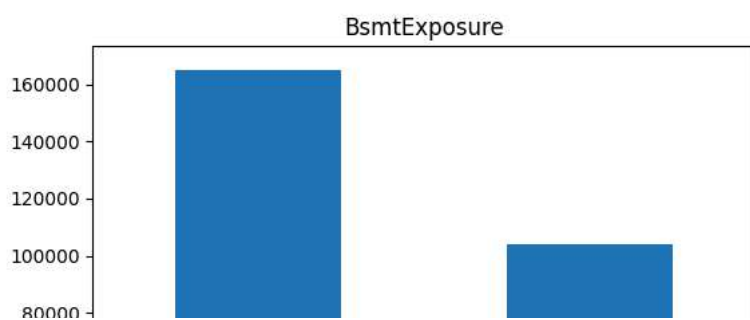
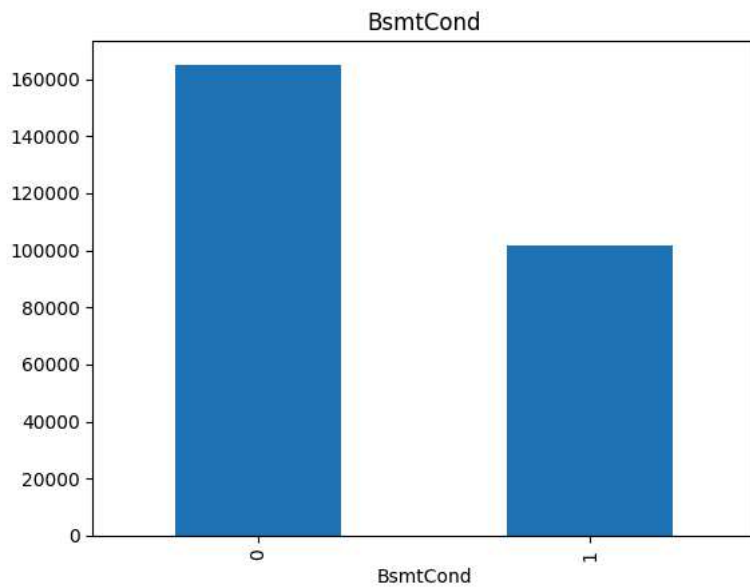
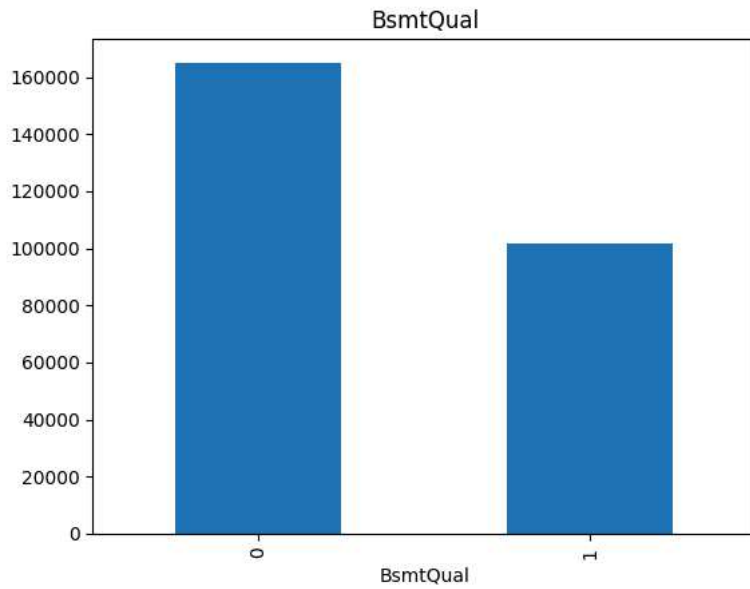
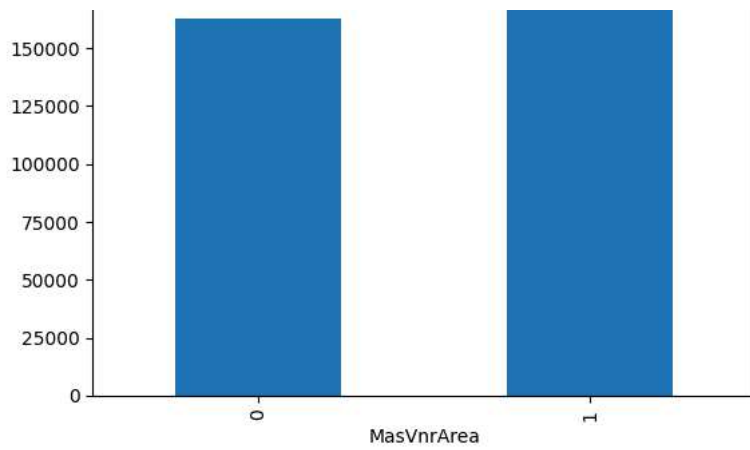
```
LotFrontage 0.1774 % missing values
Alley 0.9377 % missing values
MasVnrType 0.5973 % missing values
MasVnrArea 0.0055 % missing values
BsmtQual 0.0253 % missing values
BsmtCond 0.0253 % missing values
BsmtExposure 0.026 % missing values
BsmtFinType1 0.0253 % missing values
BsmtFinType2 0.026 % missing values
FireplaceQu 0.4726 % missing values
GarageType 0.0555 % missing values
GarageYrBlt 0.0555 % missing values
GarageFinish 0.0555 % missing values
GarageQual 0.0555 % missing values
GarageCond 0.0555 % missing values
PoolQC 0.9952 % missing values
Fence 0.8075 % missing values
MiscFeature 0.963 % missing values
```

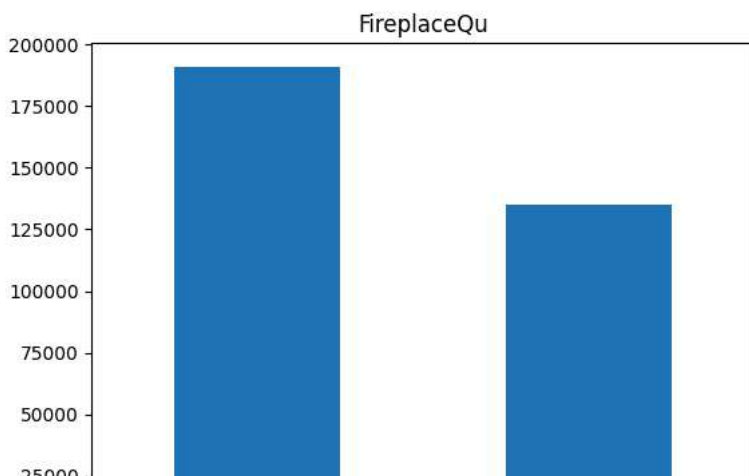
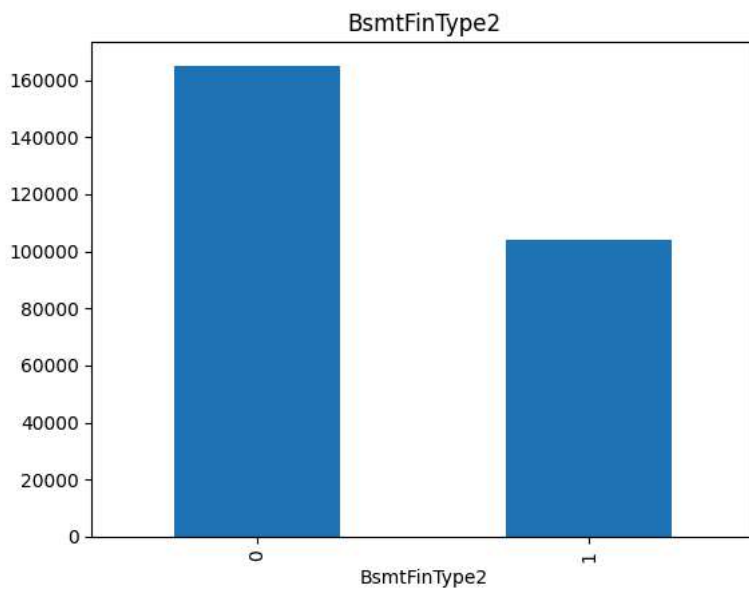
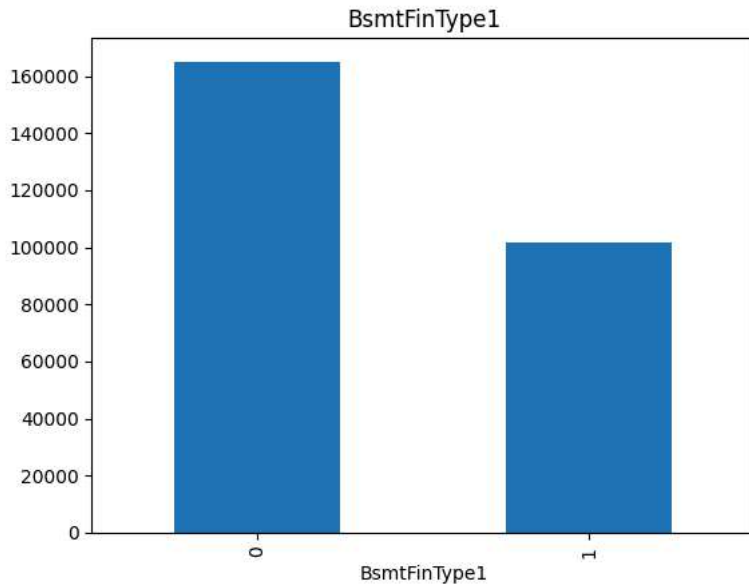
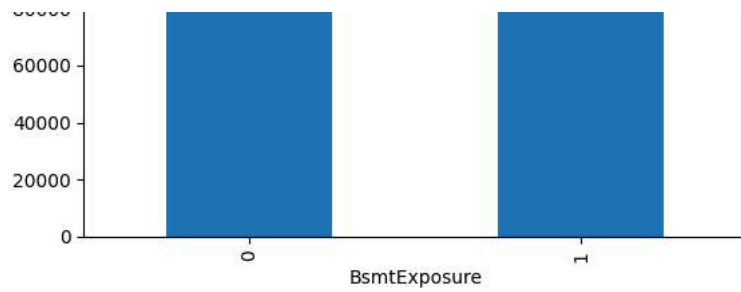
```
for feature in features_with_na:
    data = dataset.copy()

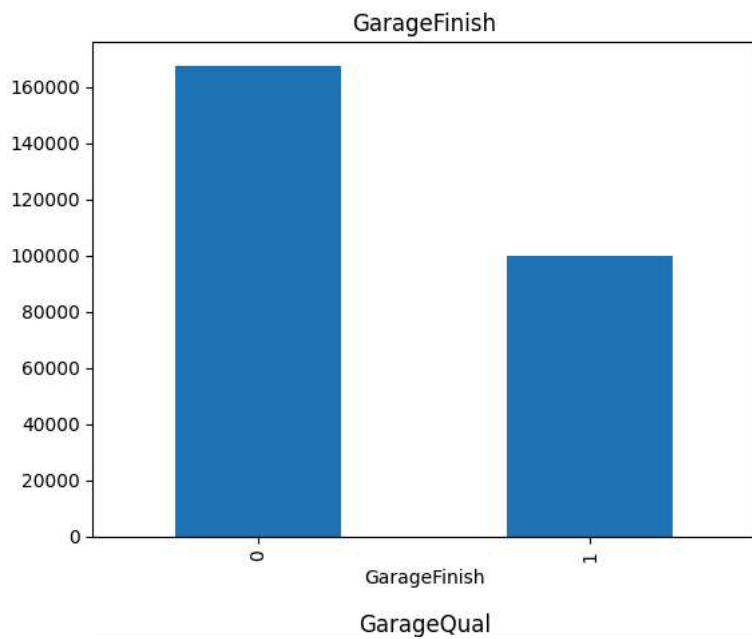
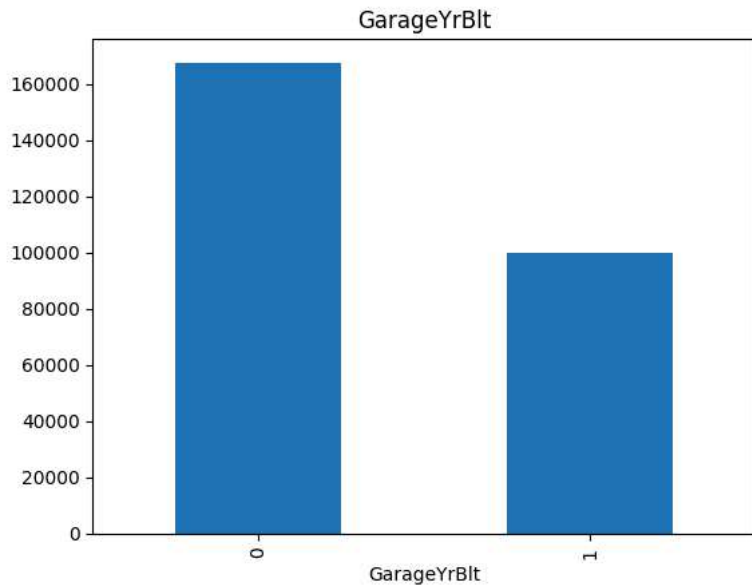
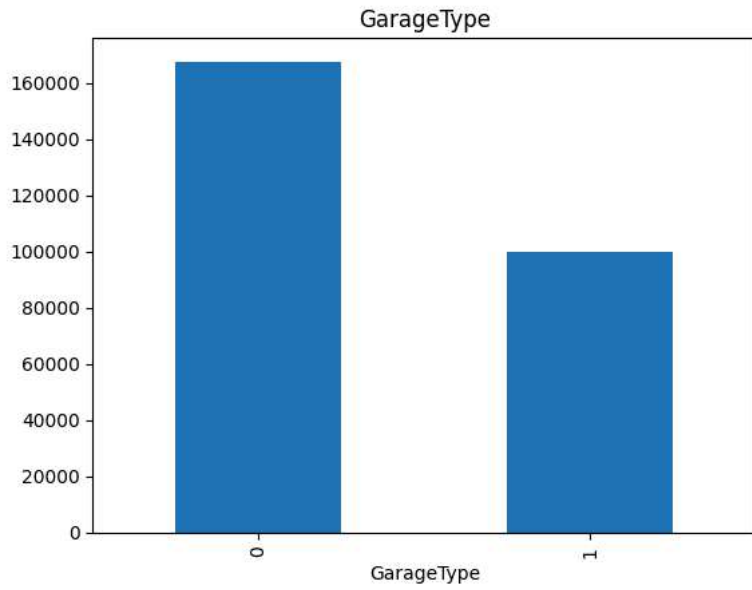
    # let's make a variable that indicates 1 if the observation was missing or zero otherwise
    data[feature] = np.where(data[feature].isnull(), 1, 0)

    # let's calculate the mean SalePrice where the information is missing or present
    data.groupby(feature)['SalePrice'].median().plot.bar()
    plt.title(feature)
    plt.show()
```

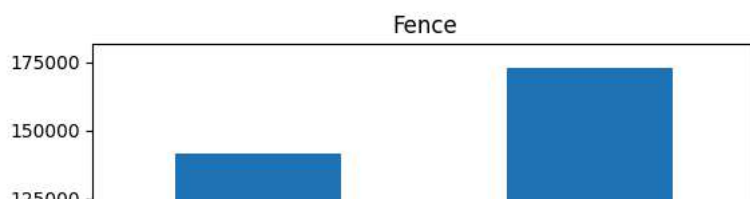
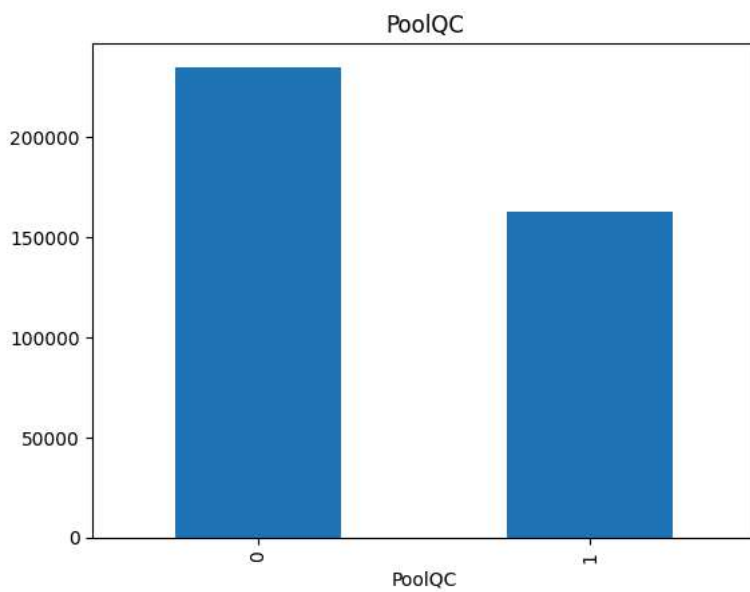
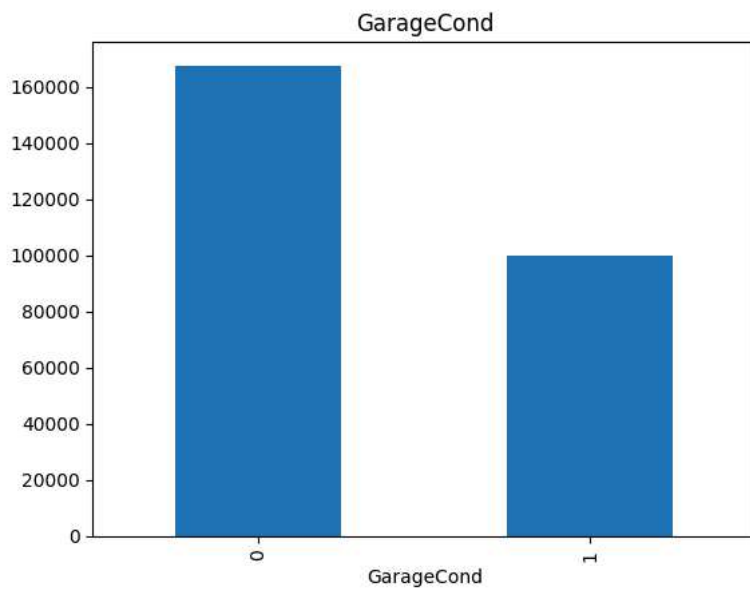
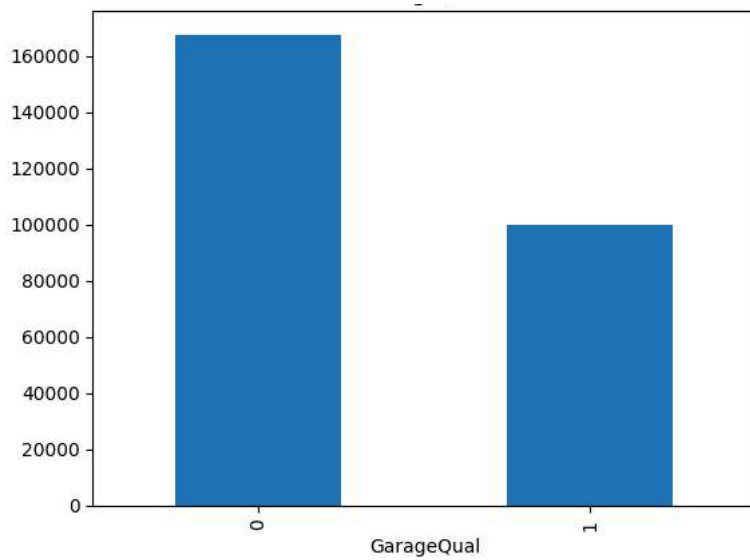


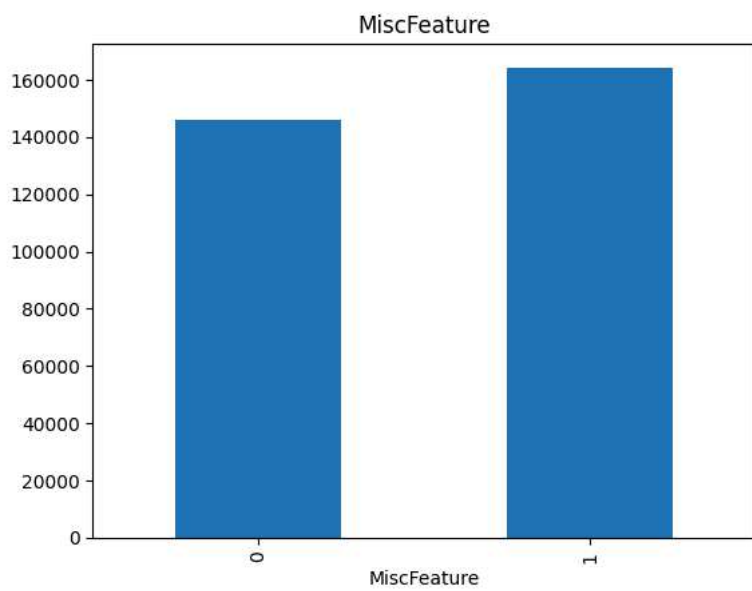
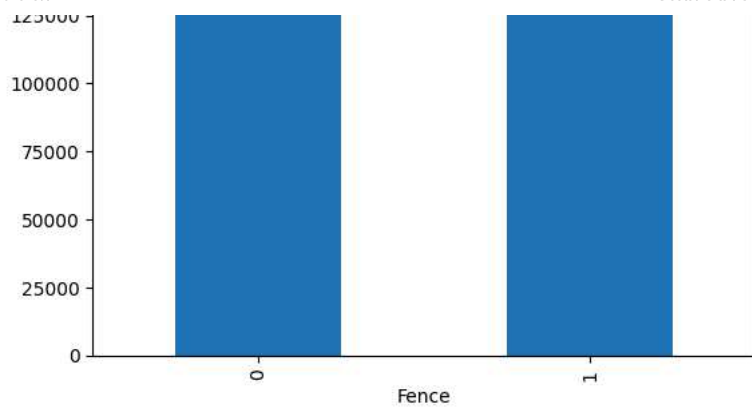






GarageQual





```
print("Id of Houses {}".format(len(dataset.Id)))
```

```
Id of Houses 1460
```

```
numerical_features = [feature for feature in dataset.columns if dataset[feature].dtypes != 'O']
```

```
print('Number of numerical variables: ', len(numerical_features))
```

```
# visualise the numerical variables
dataset[numerical_features].head()
```

```
Number of numerical variables: 38
```

	Id	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt	YearRemodAdd	MasVnrArea	BsmtFinSF1	BsmtFinSF2	BsmtUnfSF
0	1	60	65.0	8450	7	5	2003	2003	196.0	706	0	15C
1	2	20	80.0	9600	6	8	1976	1976	0.0	978	0	284
2	3	60	68.0	11250	7	5	2001	2002	162.0	486	0	434
3	4	70	60.0	9550	7	5	1915	1970	0.0	216	0	54C
4	5	60	84.0	14260	8	5	2000	2000	350.0	655	0	49C

```
year_feature = [feature for feature in numerical_features if 'Yr' in feature or 'Year' in feature]
```

```
year_feature
```

```
['YearBuilt', 'YearRemodAdd', 'GarageYrBlt', 'YrSold']
```

```
for feature in year_feature:
```

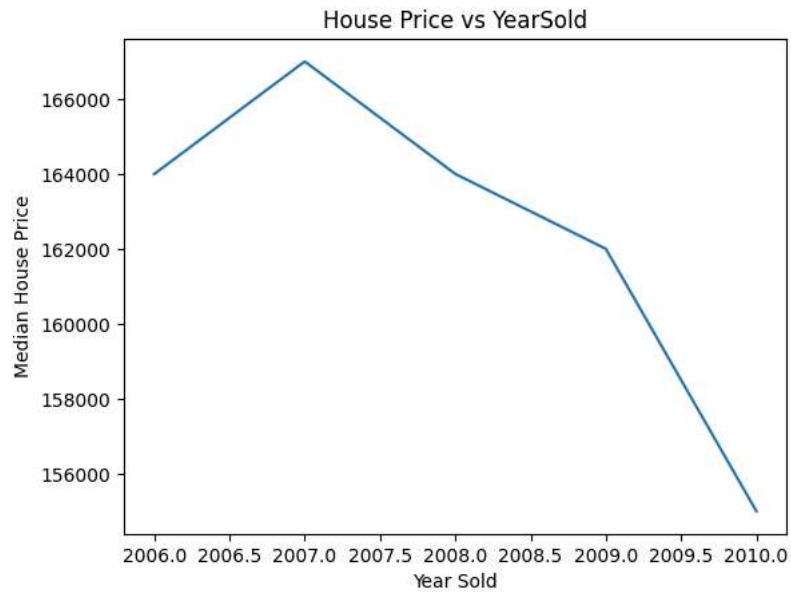
```
print(feature, dataset[feature].unique())
```

```
YearBuilt [2003 1976 2001 1915 2000 1993 2004 1973 1931 1939 1965 2005 1962 2006
1960 1929 1970 1967 1958 1930 2002 1968 2007 1951 1957 1927 1920 1966
1959 1994 1954 1953 1955 1983 1975 1997 1934 1963 1981 1964 1999 1972
1921 1945 1982 1998 1956 1948 1910 1995 1991 2009 1950 1961 1977 1985
1979 1885 1919 1990 1969 1935 1988 1971 1952 1936 1923 1924 1984 1926
1940 1941 1987 1986 2008 1908 1892 1916 1932 1918 1912 1947 1925 1900
1980 1989 1992 1949 1880 1928 1978 1922 1996 2010 1946 1913 1937 1942
1938 1974 1893 1914 1906 1890 1898 1904 1882 1875 1911 1917 1872 1905]
YearRemodAdd [2003 1976 2002 1970 2000 1995 2005 1973 1950 1965 2006 1962 2007 1960
2001 1967 2004 2008 1997 1959 1990 1955 1983 1980 1966 1963 1987 1964
1972 1996 1998 1989 1953 1956 1968 1981 1992 2009 1982 1961 1993 1999
1985 1979 1977 1969 1958 1991 1971 1952 1975 2010 1984 1986 1994 1988
1954 1957 1951 1978 1974]
GarageYrBlt [2003. 1976. 2001. 1998. 2000. 1993. 2004. 1973. 1931. 1939. 1965. 2005.
1962. 2006. 1960. 1991. 1970. 1967. 1958. 1930. 2002. 1968. 2007. 2008.
1957. 1920. 1966. 1959. 1995. 1954. 1953. nan 1983. 1977. 1997. 1985.
1963. 1981. 1964. 1999. 1935. 1990. 1945. 1987. 1989. 1915. 1956. 1948.
1974. 2009. 1950. 1961. 1921. 1900. 1979. 1951. 1969. 1936. 1975. 1971.
1923. 1984. 1926. 1955. 1986. 1988. 1916. 1932. 1972. 1918. 1980. 1924.
1996. 1940. 1949. 1994. 1910. 1978. 1982. 1992. 1925. 1941. 2010. 1927.
1947. 1937. 1942. 1938. 1952. 1928. 1922. 1934. 1906. 1914. 1946. 1908.
1929. 1933.]
YrSold [2008 2007 2006 2009 2010]
```

```
dataset.groupby('YrSold')['SalePrice'].median().plot()
plt.xlabel('Year Sold')
plt.ylabel('Median House Price')
plt.title("House Price vs YearSold")
```



```
Text(0.5, 1.0, 'House Price vs YearSold')
```

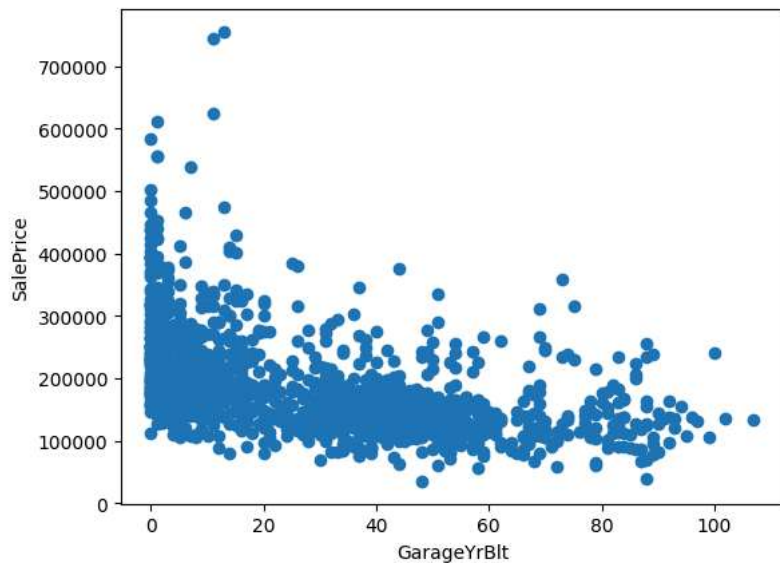
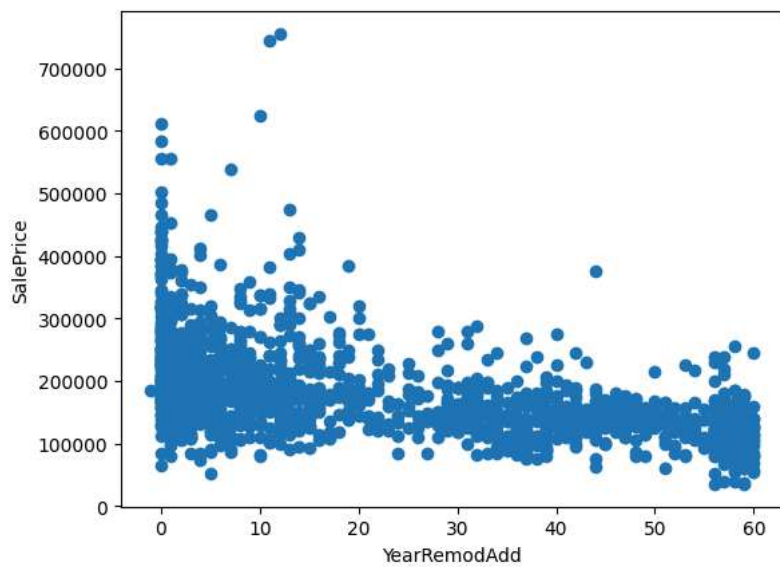
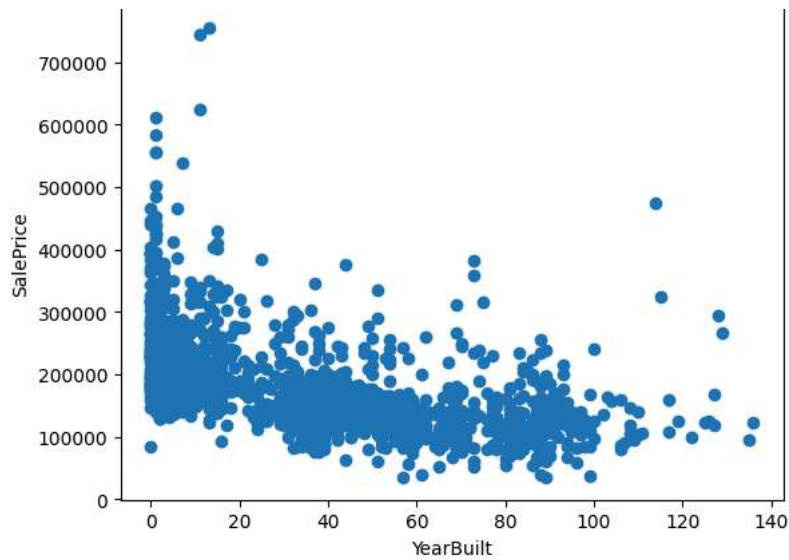


```
year_feature
```

```
['YearBuilt', 'YearRemodAdd', 'GarageYrBlt', 'YrSold']
```

```
for feature in year_feature:
    if feature != 'YrSold':
        data = dataset.copy()
        ## We will capture the difference between year variable and year the house was sold for
        data[feature] = data['YrSold'] - data[feature]

        plt.scatter(data[feature], data['SalePrice'])
        plt.xlabel(feature)
        plt.ylabel('SalePrice')
        plt.show()
```



```
discrete_feature=[feature for feature in numerical_features if len(dataset[feature].unique())<25 and feature not in year_feature+['Id']]  
print("Discrete Variables Count: {}".format(len(discrete_feature)))
```



Discrete Variables Count: 17

```
discrete_feature
```

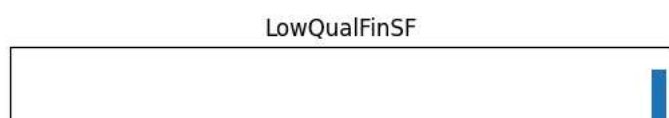
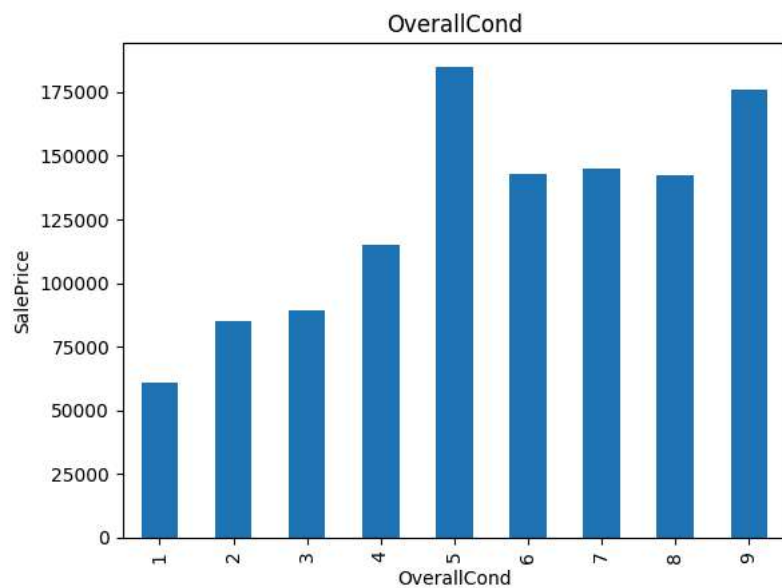
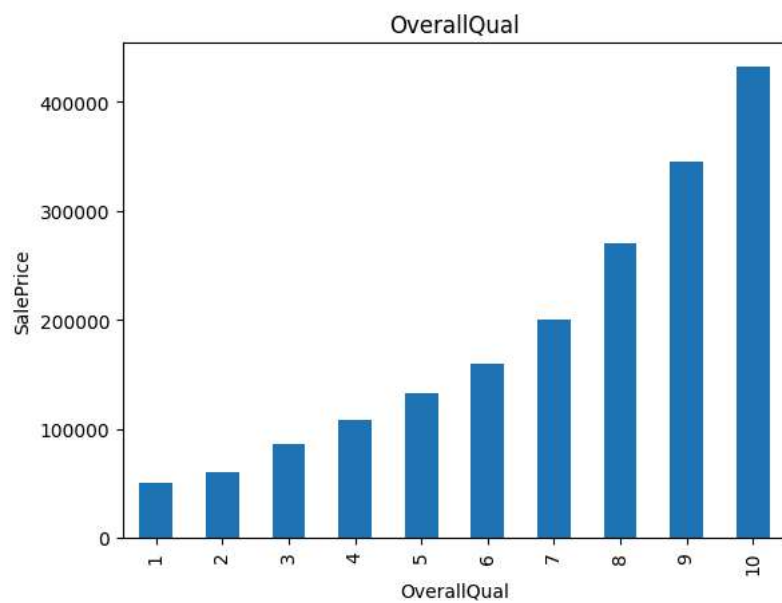
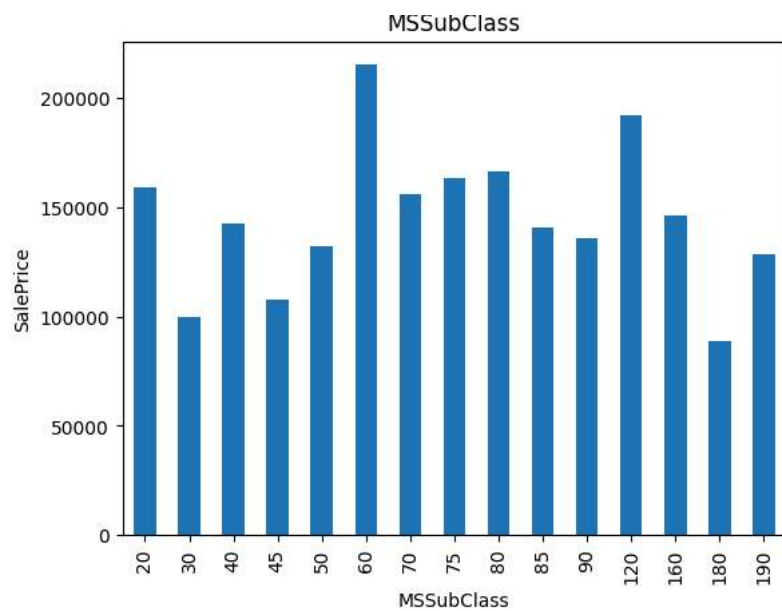
```
→ ['MSSubClass',
   'OverallQual',
   'OverallCond',
   'LowQualFinSF',
   'BsmtFullBath',
   'BsmtHalfBath',
   'FullBath',
   'HalfBath',
   'BedroomAbvGr',
   'KitchenAbvGr',
   'TotRmsAbvGrd',
   'Fireplaces',
   'GarageCars',
   '3SsnPorch',
   'PoolArea',
   'MiscVal',
   'MoSold']
```

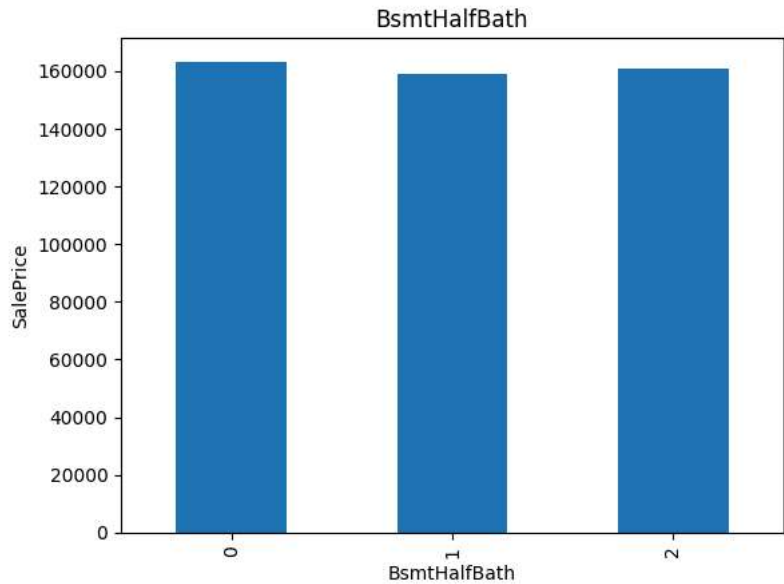
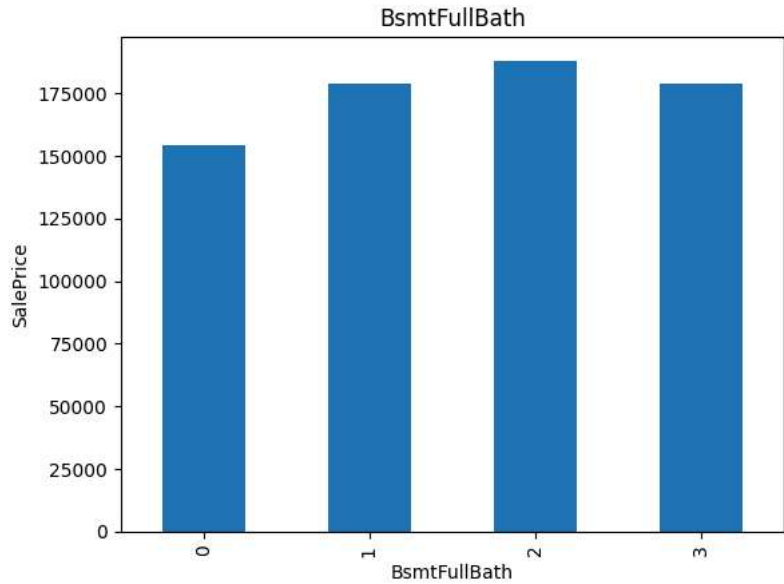
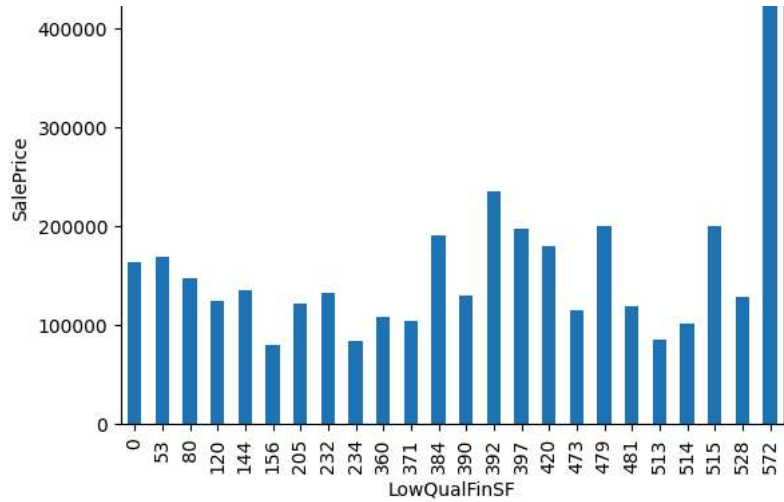
```
dataset[discrete_feature].head()
```

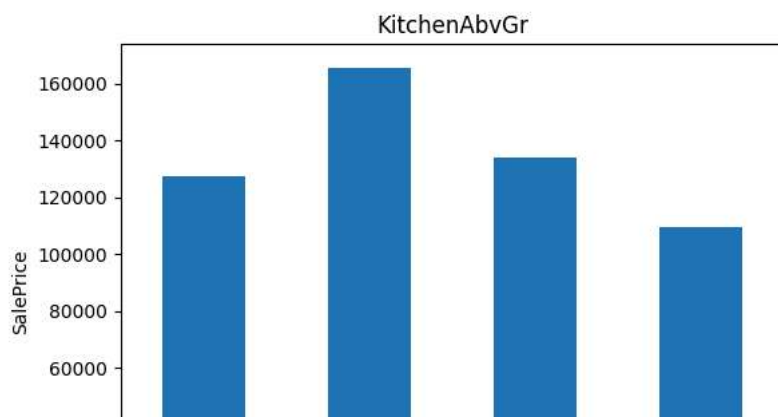
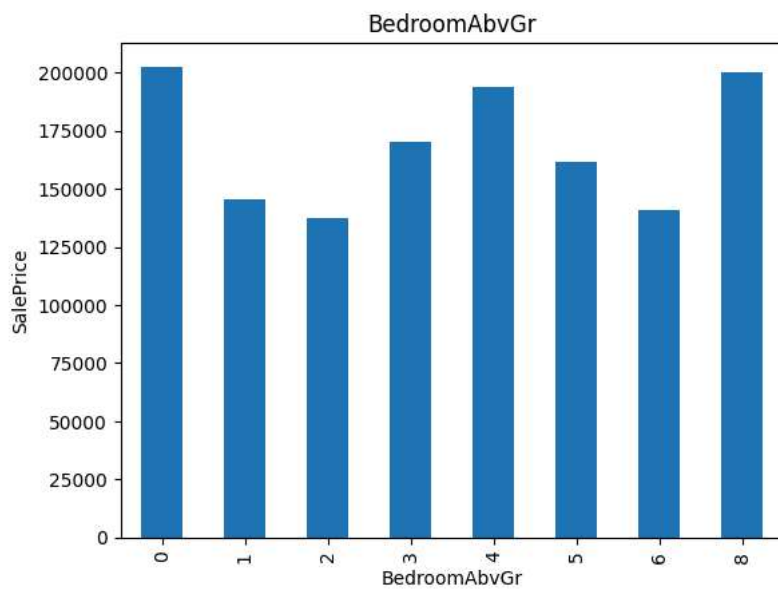
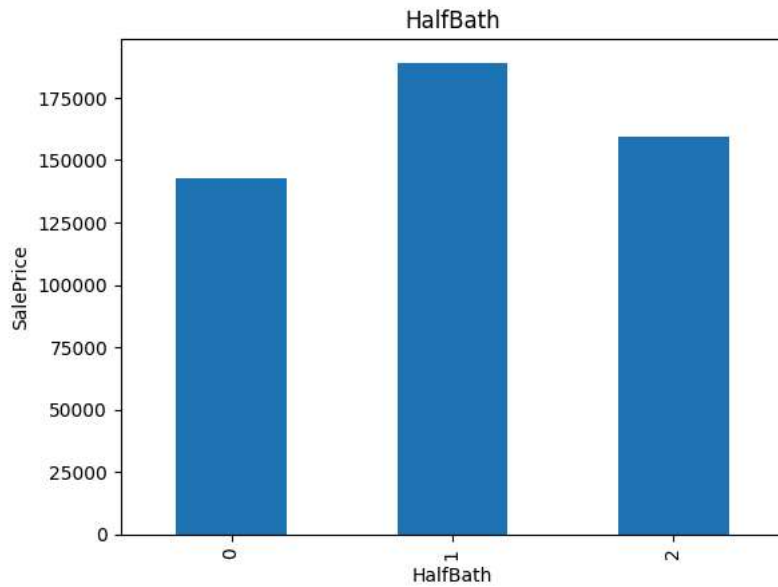
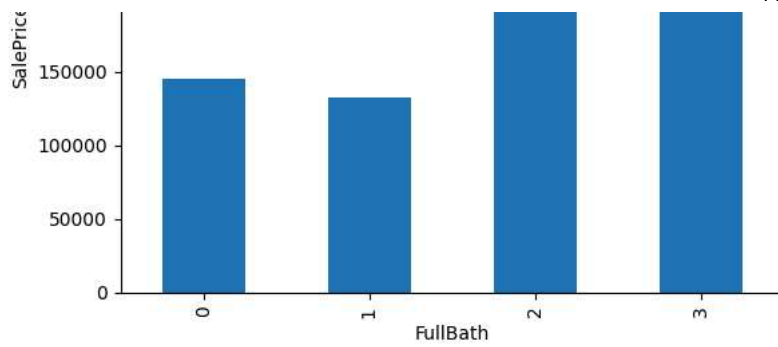
```
→
```

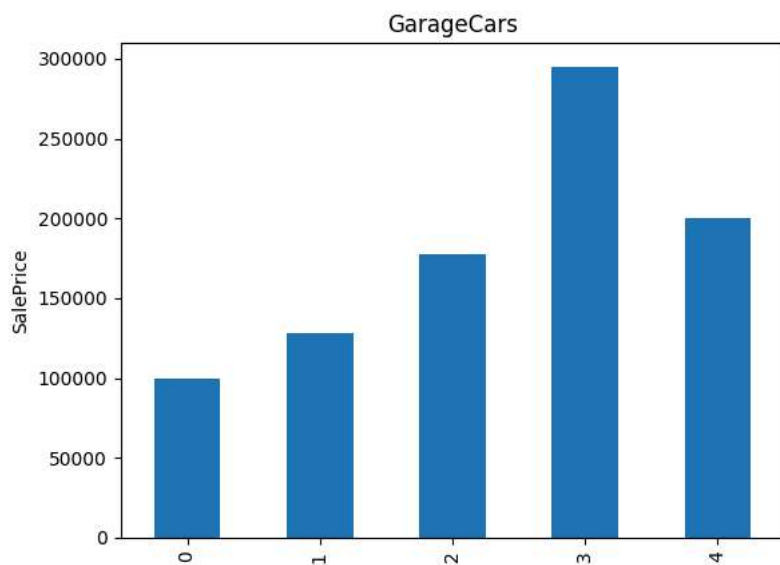
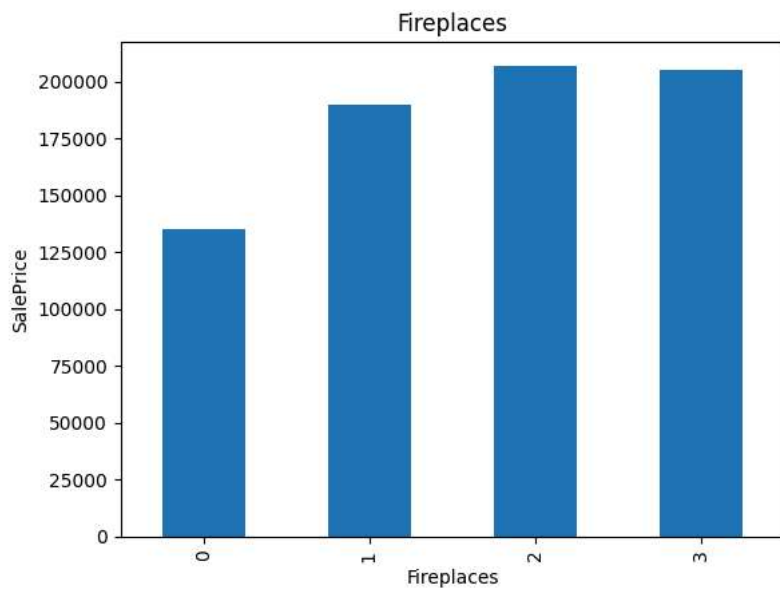
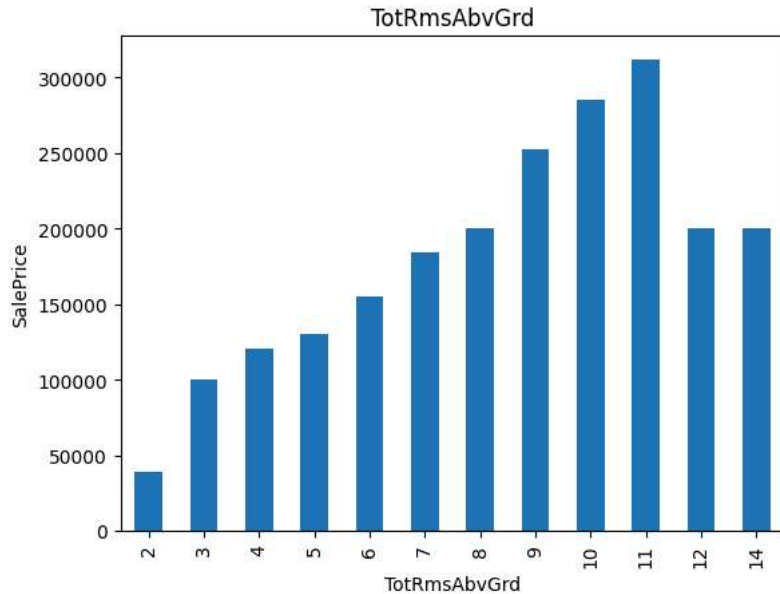
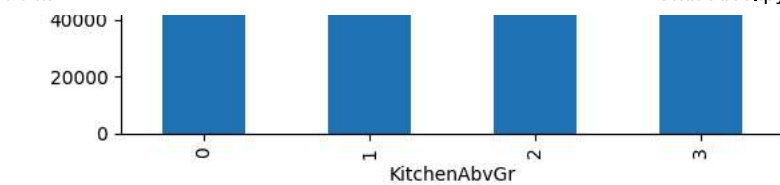
	MSSubClass	OverallQual	OverallCond	LowQualFinSF	BsmtFullBath	BsmtHalfBath	FullBath	HalfBath	BedroomAbvGr	KitchenAbvGr	TotRms
0	60	7	5	0	1	0	2	1	3	1	
1	20	6	8	0	0	1	2	0	3	1	
2	60	7	5	0	1	0	2	1	3	1	
3	70	7	5	0	1	0	1	0	3	1	
4	60	8	5	0	1	0	2	1	4	1	

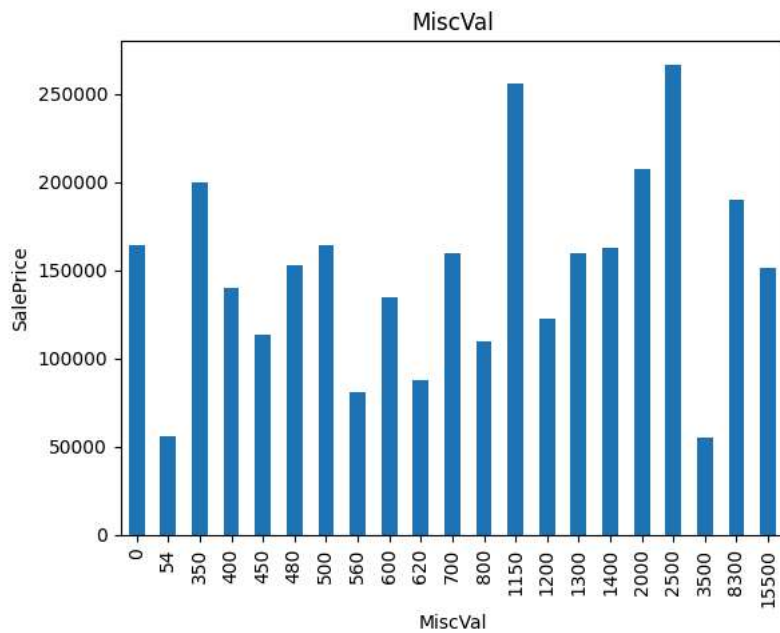
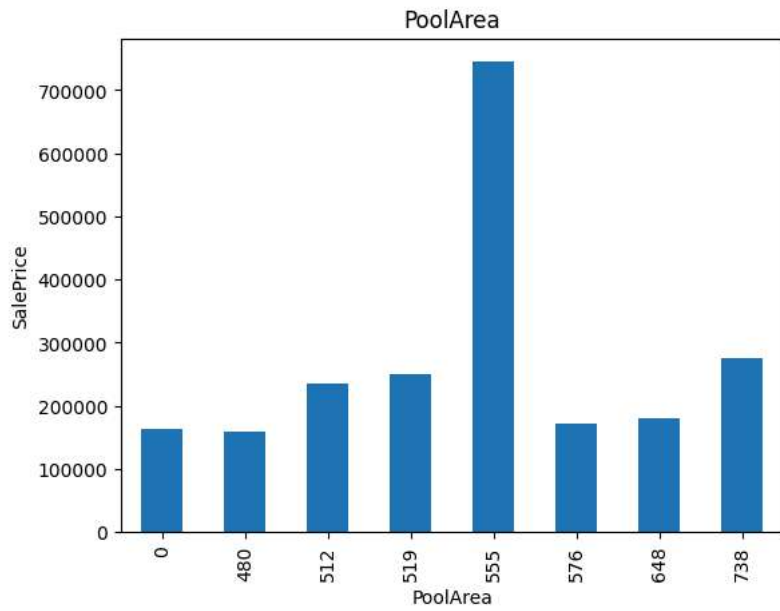
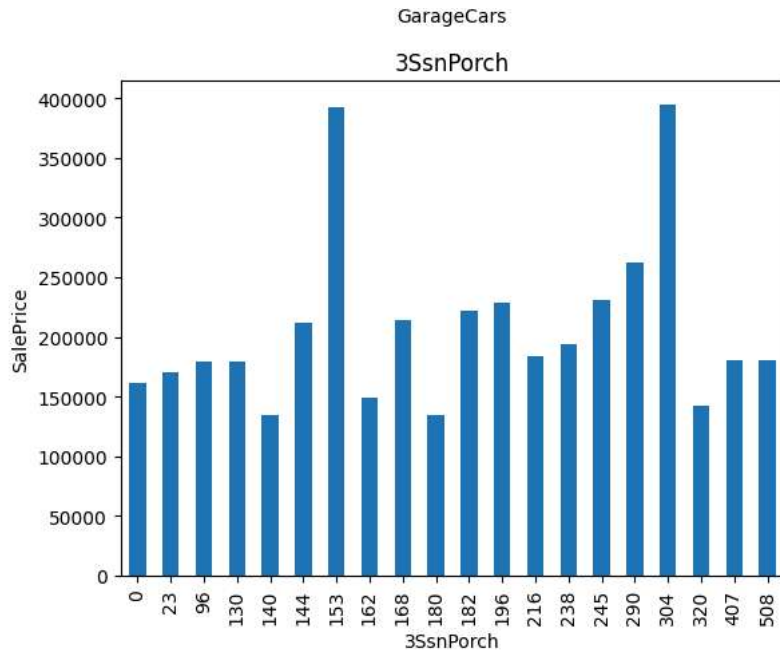
```
for feature in discrete_feature:
    data=dataset.copy()
    data.groupby(feature)['SalePrice'].median().plot.bar()
    plt.xlabel(feature)
    plt.ylabel('SalePrice')
    plt.title(feature)
    plt.show()
```



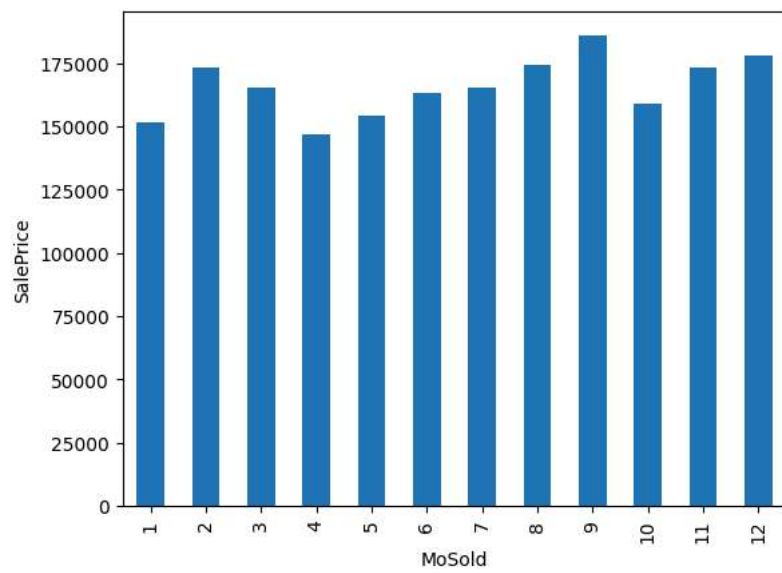








MoSold



```
continuous_feature=[feature for feature in numerical_features if feature not in discrete_feature+year_feature+['Id']]  
print("Continuous feature Count {}".format(len(continuous_feature)))
```

↔ Continuous feature Count 16

```
for feature in continuous_feature:  
    data=dataset.copy()  
    data[feature].hist(bins=25)  
    plt.xlabel(feature)  
    plt.ylabel("Count")  
    plt.title(feature)  
    plt.show()
```

